

TLaser

Real-Time Digital Twin & Parameter Calibration Platform

A Physics-Informed Neural Network Surrogate Suite for Diode Lasers

USER MANUAL & TECHNICAL REFERENCE

TARGET AUDIENCE:

Diode Laser Engineers, Optoelectronics Researchers & System Operators

AUTHOR & SERVICE SCOPE:

Zhenwen Wan (AI + Simulation Expert | Service: Custom PINN Surrogates)

1. DIGITAL TWIN ARCHITECTURE & SOLVER MAPPING

Section 1.1: Physical Device & Cavity Discretization

Telecom diode lasers emit coherent light through cleaved mirror facets. Modeling longitudinal carrier recombination requires solving coupled wave-carrier equations along the propagation z-axis cavity. The cavity is discretized into a 51-point longitudinal grid to capture spatial hole burning (SHB) depletion.

Section 1.2: 7D Geometrical Sweeps & Dataset Generation

The high-fidelity simulation sweeping tool runs random sweeps over the 7D parametric workspace: rear/front reflectivities $R1/R2$, cavity length L , ambient temperature $T0$, injection active current, active width w_{active} , and active layer thickness d_{active} . This covers 1500 samples saved in data files.

Section 1.3: Physical Rate Equation Core

The underlying core is a steady-state quasi-3D optoelectronic-thermal coupled physical simulator. It acts as the reference data synthesizer, evaluating carrier continuity, non-radiative Auger losses, and optical output power fields under high current injections.

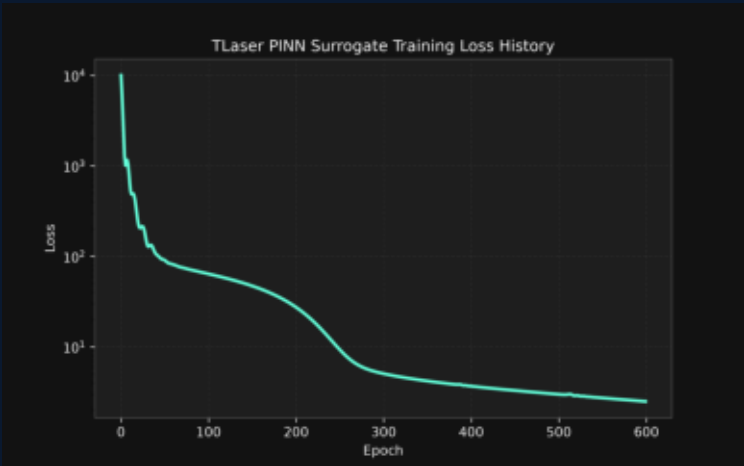


Figure 1.1: Surrogate PINN training convergence and residual loss history.

2. STREAMLIT CONTROL APP & CALIBRATION DASHBOARD

Section 2.1: Interactive Sweeps Dashboard

The dashboard (app.py) provides real-time digital twin sweeps with under 5 milliseconds latency. The sidebar controls the 7D configuration, updating 1D carrier and optical power profiles. Bilingual language toggles (English/Chinese) enable international usability.

Section 2.2: Online Parameter Calibration Engine

The calibration loop fits physical drifts by matching real-time measured Light-Current-Voltage (L-I-V) curves. The engine calibrates internal loss α_i , confinement factor Γ , Auger recombination scaling, series resistance R_s , and shunt resistance R_{sh} . Ingestion interfaces support JSON and CSV file uploads.

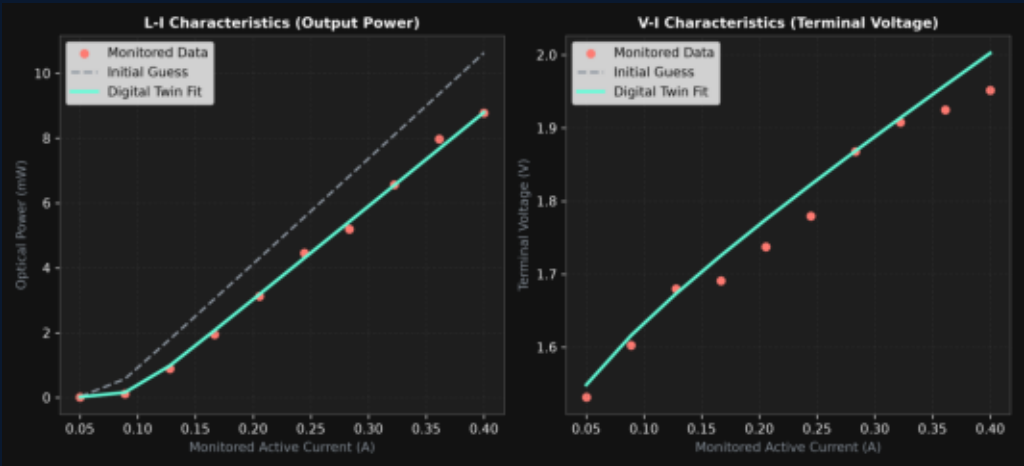


Figure 2.1: L-I and V-I curves fitting comparison before and after calibration.

3. SCIENTIFIC METHODOLOGY & FORMULATIONS

Section 3.1: Carrier Continuity Residual

The PINN training optimizer penalizes non-physical steady-state carrier deviations along the cavity grid: $G_{inj} - R_{rec}(N(z)) - R_{stim}(N(z), P(z)) = 0$. G_{inj} integrates active width and thickness: $I_{active} / (q_0 * L * w_{active} * d_{active})$. Recombination includes Shockley-Read-Hall, radiative, and temperature-dependent non-radiative Auger processes.

Section 3.2: Second-Order Photon Wave Intensity Residual

Optical waves propagation utilizes the second-order derivative approximation along the cavity: $d^2P/dz^2 - (\Gamma * g(z) - \alpha_i)^2 * P(z) = 0$. This total power residual on all internal grid nodes ensures the intensity curvature conforms to coupled longitudinal wave propagation.

Section 3.3: Neural Surrogate Architecture

The surrogate model PINNLaser uses dense layers with GELU activation. It maps the 7D parametric input vector to a 105D array representing total power, WPE, current, and longitudinal grid distributions. Input scale mappings are normalized within [0, 1] during backpropagation optimization.

TLaser Physics Constraints Model

$$\text{Loss} = \text{Loss_Data} + w_c * \text{Loss_Carrier} + w_p * \text{Loss_Photon} + w_s * \text{Loss_Smooth}$$

Enforces wave propagation & carrier continuity on 51 longitudinal grid nodes.

4. ENVIRONMENT SETUP, COMMANDS & TROUBLESHOOTING

Section 4.1: Code Compilation Commands

```
> python simulator/generate_dataset.py --num-samples 1500
> python surrogate/train.py --epochs 600
> python calibration/calibrate.py --data-file data/monitored_liv.json
> python verify_pipeline.py
> python -m streamlit run app.py
```

Section 4.2: Input Schema Specifications

JSON file structure requires fields: `current_A`, `voltage_V`, and `optical_power_W` containing arrays of floats. CSV format requires a header line, followed by three columns in order: Current (A), Voltage (V), and Power (W).

Section 4.3: Troubleshooting & Operations

For CUDA/PyTorch package errors, force-install the CPU wheel using the `whl` index parameter. If model loading fails, verify scale parameters `pinn_scale_params.npz` exist in `data/` folders. Always execute verification using `verify_pipeline.py` to validate code syntax and dataset shapes.

Zhenwen Wan (AI + Simulation Expert)

Contact for commercial collaborations or custom PINN solutions: aw4wzw@gmail.com

Project Repository: <https://github.com/ZhenwenWan/TLaser>